	Grado en Ing. Biomédica
	Sistemas Distribuidos
	Curso 2025-2026
	Práctica (proyecto)
	Gestión de recursos hospitalarios

Crear una plataforma web distribuida mediante estándares (HTML, CSS y Javascript) para la gestión de recursos hospitalarios. La aplicación permitirá a un profesional sanitario reservar los distintos recursos y dispositivos del hospital y a los gestores llevar un seguimiento y trazabilidad de todos esos recursos.

Por lo tanto, el sistema distribuido estará formado por tres módulos:

- Un **servidor** basado en Javascript mediante la plataforma NodeJS.
- Una **aplicación web para el sanitario**.
- Una **aplicación web para el gestor**.

Para la realización de la práctica se utilizarán distintos mecanismos de comunicación: Servicios Web tipo REST, RPC y WebSockets

Servidor y sistema de información

Toda la información del sistema estará centralizada en el servidor. Las dos aplicaciones obtendrán toda la información del servidor, utilizando para ello las distintas tecnologías que se proponen. Solo existirá un **UNICO servidor** con todos los datos y que implementará las TRES tecnologías.

En el servidor, los datos serán almacenados como colecciones en memoria, simples arrays de objetos de javascript (no será necesario utilizar una base de datos), por lo que cada vez que se reinicie el servidor se perderán los cambios realizados.

Los datos a gestionar son los siguientes (cada uno es un array de objetos):

- **sanitarios**: Conjunto de sanitarios del hospital
    - **id**: Identificador único de cada sanitario (1, 2, 3…)
    - **nombre**: Nombre del sanitario
    - **apellidos**: Apellidos del sanitario
    - **usuario**: Usuario utilizado en el login (es único)
    - **password**: Contraseña utilizada en el login
- **gestores**: Conjunto de gestores del hospital

    - **id**: Identificador numérico único de cada gestor (1, 2, 3…)
    - **nombre**: Nombre del gestor
    - **apellidos**: Apellidos del gestor
    - **usuario**: Usuario utilizado en el login (es único)
    - **password**: Contraseña utilizada en el login
- **ubicaciones**: Conjunto de ubicaciones (salas, almacenes, laboratorios…) donde están disponibles los recursos.
    - **id**: Identificador numérico único por cada ubicación (1,2,3…).
    - **nombre:** Nombre de la ubicación.
- **categorias:** Conjunto de categorías en las que se clasifican los recursos
    - **id:** Identificador numérico único de cada categoría (1,2,3…)
    - **nombre:** Nombre de la categoría (termómetros, estetoscopios, sillas de ruedas…)
- **modelos**: Conjunto de modelos de los recursos que se disponen. Por ejemplo distintos modelos de un termómetro, cada uno de un proveedor:
    - **id**: Identificador número único de cada modelo (1,2,3…).
    - **nombre**: Nombre del modelo.
    - **categoria**: Identificador de la categoría a la que pertenece.
    - **horas_maximas**: Horas máximas que se puede reservar un dispositivo de este modelo.
- **recurso**: Conjunto de recursos dados de alta en el sistema. De cada modelo puede haber varios recursos.
    - **id**: Identificador numérico único para cada recurso (1,2,3…).
    - **modelo**: Identificador del modelo.
    - **ubicacion:** Identificador de la ubicación donde se encuentra el dispositivo cuando está libre.
    - **numero_serie:** Número de serie del recurso
    - **estado**: Estado en el que se encuentra el recurso:
        - **0**: operativo
        - **1**: de baja o defectuoso
        - **2**: en mantenimiento
- **reservas:** Histórico de reservas de un dispositivo, tanto realizadas como pendientes.
    - **id:** Identificador numérico único para cada asignación (1,2,3…)
    - **recurso**: Identificador del recurso
    - **sanitario**: Identificador del sanitario
    - **horas_estimadas**: Horas estimadas de uso del dispositivo
    - **fecha_peticion**: Fecha/hora de la realización de la reserva
    - **fecha_inicio**: Fecha/hora de inicio de la asignación o null si aun no se ha asignado (es una reserva no iniciada)

    - **fecha_fin**: Fecha/hora de finalización de la asignación o null si no se ha finalizado aun
- **resenyas:** Histórico de reseñas realizadas por los sanitarios para indicar su experiencia de uso con el recurso.
    - **Id:** Identificador numérico único de la reseña (1, 2, 3…)
    - **recurso:** Id del recurso sobre la que se hace la reseña.
    - **sanitario:** Sanitario que realiza la reseña
    - **fecha:** Fecha de la reseña
    - **valor:** Evaluación de la reseña (valor de 1 a 5)
    - **descripcion:** Texto descriptivo de la reseña

Todas las fechas o fechas/hora se almacenarán en el servidor como un objeto **Date** de Javascript.

A continuación, se describen los requerimientos funcionales de cada uno de los módulos

Parte 1: App del gestor

La primera parte de la práctica consiste en la realización de la aplicación del gestor.

Se trata de una aplicación web que está alojada y trabaja con el servidor, e implementada con servicios web de tipo **REST asíncronos**.

Requerimientos funcionales

Los requerimientos funcionales de la aplicación del gestor son:

- El gestor entrará al sistema mediante un acceso por usuario y contraseña. Sólo si la combinación usuario/contraseña es correcta podrá continuar.

- Si el usuario no se ha registrado aún podrá pulsar el botón “Registrar” para darse de alta. Se le pedirán los datos del gestor.

- Una vez dentro del sistema:

  - Se le mostrará en la vista principal una bienvenida con su **nombre y apellidos**
  - Tendrá una opción que le permitirá **salir** del sistema (la aplicación vuelve a la pantalla de login).
  - Tendrá un botón para editar los datos del gestor. Se puede usar la misma vista que el registro de gestor.
  - Un listado/tabla de los recursos creados en el sistema.
    - Se mostrará por cada recurso:
      - Número de serie
      - Categoría
      - Modelo
      - Ubicación
      - Estado

- Para ayudar a localizar un recurso para obtener un listado con datos hay que realizar una búsqueda (inicialmente, al entrar, el listado estará vacío). Para buscar se seleccinonará un conjunto de criterios (categoría, modelo, ubicación y estado) y se pulsará sobre el botón 'Buscar'. Todos los criterios pueden estar con el valor "Todos", que indica que no se quiere filtrar por ese criterio.
- Cada recurso tendrá un botón para borrar ese recurso.
- Cada recurso tendrá un botón para abrir la ficha del recurso
- Pulsando 'Añadir Recurso" se podrá dar de alta un nuevo recurso.

- Tanto al crear un recurso como al abrir un existente se abrirá una ficha con la información del recurso:

d Recurso

Aplicación Gestor

RECURSO

Datos recurso

Categoría: Termómetro
Modelo: ASY-234
Núm. Serie: AD58754
Ubicación: Almacen 1
Estado: Operativo

Guardar

Reservas

Sanitario	H. estim.	F. Petición	F. Inicio	F. Fin
María Estrella López	2	12/04/25 14:30:50	12/04/25 14:40:00	12/04/25 16:00:00
Luis Navarro García	2	13/04/25 10:00:00	13/04/25 10:30:00	
Amparo Maciá Ruiz	2	13/04/25 10:00:00		

Reseñas

Fecha	Sanitario	Valoración	Comentario
11/04/25 13:52:00	María Estrella López	5	Todo perfecto

Volver al listado c

- La ficha tendrá los datos del recurso que se podrán editar:
  - Categoría (se elige del listado de categorías)
  - Modelo (se elige del listado de modelos)
  - Número de serie
  - Ubicación (se elige del listado de ubicaciones)
  - Estado
- Tendrá un botón guardar que guarda los datos del recurso (o lo crea si es un nuevo recurso)

- Se mostrará un listado con todas las reservas realizadas con los datos:
  - Nombre del sanitario
  - Horas estimadas de uso
  - Fecha de petición
  - Fecha de inicio
- Cada fila de las reservas será de un color indicando:
  - Blanco: reserva finalizada
  - Azul: el recurso está en uso y no se han superado las horas estimadas
  - Rojo: el recurso está en uso y se han superado las horas estimadas
  - Verde: la reserva está pendiente de que se libere el recurso
- Tendrá un listado de reseñas con los siguientes datos:
  - Fecha de la reseña
  - Nombre del sanitario que la realizó
  - Valoración
  - Comentario
- Tendrá un botón para volver al listado de recursos.

Esta gestión se implementará mediante la tecnología de Servicios Web tipo **REST asíncronos** entre el servidor y la aplicación del gestor.

Servidor REST

Para que la aplicación del gestor obtenga toda la información necesaria, el servidor ofertará un conjunto de servicios REST con Node/Express, para acceder a la información almacenada en los arrays del servidor.

Los servicios a implementar son (SOLO IMPLEMENTAR ESTOS, **NO SE PUEDE IMPLEMENTAR OTROS)**:

URL	Método	Descripción
/api/ubicaciones	GET	Obtiene un array con todas las ubicaciones
/api/categorias	GET	Obtiene un array con todas las categorías
/api/modelos	GET	Obtiene un array con todas los modelos

/api/gestores/login	POST	Realiza un login para gestor.
		En body se indican las credenciales. Por ejemplo:
		`{`
		`  "login": "xxx",`
		`  "password": "secreto"`
		`}`
		Si va bien se obtiene el id (un número) del gestor:
		5

/api/gestores	POST	Crea un nuevo gestor (registro). Se le pasa un objeto con los datos del gestor (excepto id). El login tiene que ser único.
/api/gestores/:id	PUT	Actualiza los datos del gestor. Se le pasa un objeto con los datos a actualizar del gestor (excepto id). El login tiene que ser único.
/api/gestores/:id	GET	Obtiene un objeto con los datos de un gestor (no devolver contraseña)
/api/sanitarios/:id	GET	Obtiene un objeto con los datos de un sanitario (no devolver el código de acceso)
/api/recursos? categoria=idCategoria& modelo=idModelo& ubicación=idUbicacion& estado=idEstado	GET	Obtiene todos los recursos. Opcionalmente se puede filtrar por algunos criterios.
/api/recursos/:id	GET	Obtiene los datos de un recurso
/api/recursos	POST	Crea un recurso
/api/recursos/:id	PUT	Actualiza
/api/recursos/:id	DELETE	Borra un recurso
/api/recursos/:id/reservas	GET	Obtiene las reservas de un recurso
/api/recursos/:id/resenyas	GET	Obtiene los reseñas de un recurso

Todos los servicios retornarán como código de estado:

- 200: Si todo ha ido bien
- 201: Si un servicio POST ha creado correctamente un registro
- 404: Si se solicita un elemento que no existe (p.e. GET /api/gestor/7, y no existe un gestor con código 7). No usar en los listados cuando es un array vacio, en ese caso 200 y array vacío.
- 403: Si la autenticación (login) no es correcta

Parte 2: App del sanitario

En esta segunda parte del proyecto habrá que desarrollar la aplicación que usará el sanitario.

Requerimientos funcionales

- El sanitario entrará al sistema mediante un acceso por usuario y contraseña. Sólo si la combinación usuario/contraseña es correcta podrá continuar.

a Acceso (Login)

- Si el sanitario no se ha registrado aún podrá pulsar el botón “Registrar” para darse de alta. Se le pedirán los datos del sanitario.

b Registro / Modificación datos

- Una vez dentro del sistema:

Si el acceso es correcto, se mostrará una ventana de inicio con:

- El nombre y apellidos del sanitario
- Un botón para editar los datos del sanitario (se puede usar la vista de registro)
- Un listado de las reservas pendientes (cuando aún no ha sido retirado el recurso) con:
    - Categoría
    - Modelo
    - Número de serie
    - Fecha de petición
    - Horas restantes: Total de horas restantes para la devolución del recurso por otros sanitarios que lo han reservado
    - Botón de retirar (sólo si tiempo es 0 y está devuelto)
    - Botón de cancelar reserva
- Un listado con las reservas realizadas con:
    - Categoría
    - Modelo
    - Número de serie
    - Horas estimadas de uso
    - Fecha/Hora de inicio (recogida del recurso)
    - Fecha/Hora de fin (devolución del recurso)
- Botón para crear una nueva reserva
- Botón de salir que nos lleva al login

c Inicio (Home)

Aplicación Sanitario

INICIO

Bienvenido **María Gómez Gómez** Editar datos b

Reservas pendientes

Categoria	Modelo	N.Serie	F.Petición	H.Rest.		
Termómetro	ASY-234	et785	12/04/25 16:00:00	2.5		X
Termómetro	ASY-234	AD58754	12/04/25 16:00:00	0	Retirar	X

Reservas realizadas

Categoria	Modelo	N.Serie	H.E.	F.Ini	F.Fin	
Termómetro	ASY-234	AD58754	2	10/04/25 16:30:00	10/04/25 18:30:00	Reseña
Termómetro	ASY-234	AD58754	2	12/04/25 14:00:00	12/04/25 16:00:00	Reseña
Termómetro	ASY-234	XD7815	2	13/04/25 10:00:00	Devolver	Reseña

Nueva reserva f

Salir a

Cuando se pulsa en nueva reserva se abre la siguiente vista:

d Nueva Reserva

Aplicación Sanitario

NUEVA RESERVA

Categoría: Termómetro

Modelo: ASY-234

T. est. uso: 1

N.Serie	H.Rest.	Valoración	
AD58754	Disponible	4	Retirar
UI983	7.2	3.2	Reservar
HJ9487	Disponible	5	Retirar

Cancelar c

- Categoría a filtrar
- Modelo a filtrar (de la categoría seleccionada)
- Tiempo estimado de uso del recurso reservado
- Listado de recursos en función del modelo elegido (solo recursos en estado operativos)
    - Número de serie
    - Horas restantes para que el recurso esté disponible (según otras reservas) o la palabra “Disponible”
    - Valoración media del recurso por las reseñas de los sanitarios

  - Un botón de “Reservar” si el recurso está ocupado
  - Un botón de “Retirar” si el recurso está libre
- Un botón para volver al inicio

Para añadir una reseña desde el listado de reservas realizadas se abrirá la siguiente vista:

- Número de serie del recurso que se está reseñando
- Valoración numérica
- Texto de la valoración

Servidor

El servidor RPC implementará todas las funciones que podrá invocar el cliente. Utilizar la librería RPC que se aporta.

Hay que implementar los siguientes procedimientos RPC:

Nombre	Argumentos	Descripción y Retorno
`obtenerCategorias`		Array con todas las categorías
`obtenerModelos`		Array con todos los modelos
`loginSanitario`	`usuario` `password`	Valida el código de acceso del usuario. Retorna el id del sanitario si es correcto o null si no es correcto
`crearSanitario`	`datosSanitario`	Crea un nuevo sanitario. Se le pasa un objeto con todos los datos del sanbitario. Si todo está bien retorna el código del nuevo sanitario. Si hay algún error retorna null. NO SE PUEDE REPETIR EL LOGIN
`actualizarSanitario`	`idSanitario` `datosSanitario`	Actualiza los datos de un sanitario. NO SE PUEDE REPETIR EL LOGIN.
`obtenerSanitario`	`idSanitario`	Obtiene un objeto con los datos del sanitario (excepto password) o null si no existe.
`obtenerRecursos`	`idModelo`	Obtiene un array de objetos de recursos de un modelo concreto. Solo

		recursos que su estado es operativo
`tiempoPendiente`	`idRecurso`	Calcula el tiempo restante para que el recurso esté disponible o 0 si ya está disponible. Para ello se contempla el tiempo estimado de uso del usuario que lo tenga y otras reservas previas.
`obtenerReservas`	`idSanitario`	Obtiene las reservas de un sanitario
`obtenerResenyas`	`idRecurso`	Obtiene las reseñas de un recurso

`crearResenya`	`idRecurso`
	`idSanitario`
	`valoracion`
	`descripcion`
		Registra una nueva reseña. Retorna el id de la reseña o null.

`reservarRecurso`	`idRecurso`	Reserva un recurso y retorna el id de la reserva o null
`cancelarReserva`	`idReserva`	Cancela una reserva
`iniciarReserva`	`idReserva`	Inicia la reserva (retirada del recurso)
`finalizarReserva`	`idReserva`	Finaliza la reserva (devolución del recurso)

Integrar este servidor con el servidor de REST, de forma que se lance un único servidor. Se recomienda separar en archivos JS distintos el servidor principal, el servidor REST, el servidor RPC y los datos (que se usaran desde los dos servidores).

NOTAS

- En la web, **ni mostrar ni solicitar en ningún momento ningún ID**. Son valores internos que se usan en la programación pero que al usuario no le interesa.
- **No será necesario realizar una gestión de las ubicaciones, categorias y modelos** . Estos listados ya estará creado de forma predefinida en sus arrays al iniciar el servidor.
- Para agilizar el desarrollo se recomienda tener el resto de los arrays con ciertos datos (gestores, sanitarios, recursos …) que faciliten las pruebas.